



Vivekanand Education Society's Institute of Technology

(Autonomous Institute Affiliated to University of Mumbai, Approved by AICTE & Recognised by Govt. of Maharashtra)

NAAC accredited with 'A' grade

PROJECT REPORT ON

NovaScan: AI-Driven DevSecOps Platform

SUBMITTED IN FULFILLMENT OF THE REQUIREMENT FOR
SEMESTER VI OF

T.E.(Information Technology)

SUBMITTED BY

Mr Suhas Arali (01)

Mr Rohan Bhangale (03)

Mr Ayush Karanth (10)

Mr Prakhar Shrivastava (19)

UNDER THE GUIDANCE OF

Prof. Dr. Rupali Kale

V.E.S. INSTITUTE OF TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

2025-26



Vivekanand Education Society's Institute of Technology

(Autonomous Institute Affiliated to University of Mumbai, Approved by AICTE & Recognised by Govt. of Maharashtra)
NAAC accredited with 'A' grade

Certificate

This is to certify that project entitled

NovaScan: AI-Driven DevSecOps Platform

Group Members Names

Mr Suhas Arali (01)
Mr Rohan Bhangale (03)
Mr Ayush Karanth (10)
Mr Prakhar Shrivastava (19)

This project is approved in partial fulfillment of the requirements for the degree of Bachelor of Engineering (Semester VI) in Information Technology

Prof. Dr. Rupali Kale

External Examiner

Dr. (Mrs.) Shalu Chopra H.O.D

Dr. (Mrs.) J.M.Nair Principal

Date: / /2026

Place: VESIT, Chembur

College Seal

Declaration

I declare that this written submission represents my ideas in my own words. Wherever the ideas or words of others have been included, I have adequately cited and referenced the original sources. I further declare that I have adhered to all principles of academic honesty, integrity, and ethical conduct in the preparation of this work.

I affirm that I have neither misrepresented nor fabricated nor falsified any idea, data, fact, or source in my submission. I have ensured that all information presented is accurate to the best of my knowledge and obtained through ethical means.

I understand that any violation of the above principles may lead to disciplinary action by the Institute and may also invite penal action from the concerned sources if proper citation or permission has not been provided where required.

(Signature)

Suhas arali (01)

Rohan Bhangale (03)

Ayush Karanth (10)

Prakhar Shrivastava (19)

Abstract

The rapid acceleration of the Software Development Life Cycle (SDLC) through Agile and DevOps methodologies has inadvertently created severe administrative and cognitive bottlenecks for engineering teams. Developers face alert fatigue from rigid static security scanners, and Project Managers suffer from 'Agile Administration Friction,' spending excessive hours manually transcribing standup meetings into project management tools. This project introduces NovaScan, an AI-driven project intelligence and DevSecOps platform designed to eliminate these inefficiencies.

NovaScan employs a dual-engine architecture built on an asynchronous FastAPI backend and MongoDB. The first engine—the Automated Security Engineer—intercepts GitHub webhooks to analyze raw code diffs via Large Language Models (LLMs), generating dynamic risk scores and dispatching alerts to Slack, Jira, and Notion. The second engine—the Autonomous AI Scrum Master—ingests unstructured meeting transcripts, performs entity resolution against a database roster, and extracts actionable tasks into strict JSON formats for automated Jira ticket provisioning.

In the current phase, the core backend architecture has been designed and its foundational modules have been developed and partially tested. The GitHub webhook ingestion pipeline, the LLM-based risk scoring engine, and the MongoDB schema have been implemented. The Next.js frontend dashboard is in its initial stages, providing a basic interface for viewing security events. Work on the standup processing engine and multi-platform fan-out layer is ongoing.

Keywords: *DevSecOps, Artificial Intelligence, LLM, FastAPI, GitHub Webhooks, Agile Automation, Risk Scoring, NLP, MongoDB, Jira Integration*

Contents

1.	Introduction	Page No.
1.1	Introduction to the project	10
1.2.	Objectives	10
1.3.	Motivation	11
1.4.	Brief Introduction to Literature	11
1.5.	Problem Definition	11
1.6.	Scope of the Work	12
1.7.	Feasibility study	12
1.8.	SDG Mapped, Societal Impact and benefits	12
1.9.	Organisation of the Report	12
2.	Literature survey	14
2.1.	Introduction	14
2.2	Related work (Minimum 5 papers with Abstract and inference drawn)	14
2.3.	Comparative Analysis of existing related work	15
2.4.	Research Gaps Identified	16
3.	Design Implementation	17
3.1.	Introduction	17
3.2.	Nonfunctional and Functional Requirement	17
3.3.	Proposed Design & Algorithm	18
3.4.	Architectural Diagrams 3.4.1. Use Case Diagram 3.4.2. Block Diagram	18

3.5.	Proposed Planning 3.5.1. Timeline chart (Gantt chart) 3.5.2. Cost estimation	21
3.6.	Hardware requirement and Software requirements	22
4	Proposed Results and Discussions	24
4.1.	Data Collection, Acquisition, Data Preprocessing & Data Interpretation	24
4.2.	Experimental setup	24
4.3.	Evaluation Metrics	25
4.4.	Comparison of the Existing system with the Proposed Systems	25
5	Plan of action for the next semester	26
5.1.	Work done till date	26
5.2.	Plan of action for Project II	26
6	Conclusion	28
7	References IEEE format (Minimum 15-20)	29
8	Appendix	30
8.1.	Paper publications: acceptance mail, Published Paper, Plagiarism	27

List of Figures

Sr. No.	Title	Page No.
3.1	Frontend Architecture Diagram	19
3.2	Backend Architecture Diagram	19
3.3	Use Case Diagram	20
3.4	Data Flow Diagram (Level 1)	20
3.5	System Block Diagram	21

List of Table

Sr. No.	Title	Page No.
2.1	Comparative Analysis of Existing Tools and NovaScan Enhancements	15
3.1	Project Development Timeline and Phase-wise Progress	21
3.2	Cost Estimation of System Resources and Services	22
3.3	Software Requirements and Technology Stack Overview	23
4.1	Evaluation Metrics of NovaScan System (Current Phase)	25
4.2	Comparison of Existing System (SonarQube) and Proposed System	25

ACKNOWLEDGEMENT

The project report on NovaScan: AI-Driven DevSecOps Platform is the outcome of the guidance, moral support and devotion bestowed on our group throughout our work. For this we acknowledge and express our profound sense of gratitude to everybody who has been the source of inspiration throughout project preparation. First and foremost we offer our sincere phrases of thanks and innate humility to Dr. (Mrs.) Shalu Chopra H.O.D, Dr. (Mr) Manoj Sabnis Deputy H.O.D, Dr. Rupali Kale for providing the valuable inputs and the consistent guidance and support provided by them. We can say in words that we must at the outset tender our intimacy for receipt of affectionate care to Vivekanand Education Society's Institute of Technology for providing such a stimulating atmosphere and conducive work environment.

Chapter 1

Introduction

NovaScan is an AI-driven DevSecOps and agile automation platform designed to reduce inefficiencies in modern software development workflows. It integrates with tools like GitHub, Jira, and Slack to automate code review and project management tasks. Using an LLM-based engine, it analyzes pull requests to generate contextual risk scores and converts meeting transcripts into actionable Jira tickets. The system addresses key challenges such as alert fatigue, administrative overhead, and fragmented tool ecosystems.

1.1 Introduction to the Project

Modern software engineering is characterized by continuous integration, continuous deployment (CI/CD), and rapid agile iterations. As engineering velocity increases, the cognitive and administrative overhead required to maintain these workflows scales disproportionately. A typical software team today operates across a fragmented ecosystem of tools—GitHub for source control, Jira for project tracking, Slack for communication, and Google Meet for standups—yet none of these tools communicate intelligently with one another.

NovaScan is an AI-native platform engineered to reclaim this lost engineering time. Rather than acting as a passive dashboard or a rigid static analysis tool, NovaScan operates as an autonomous, asynchronous agent. By deeply integrating into the fabric of daily engineering tools, NovaScan serves two distinct roles: a vigilant DevSecOps Code Reviewer and an Autonomous AI Scrum Master. It translates unstructured human output—both raw code diffs and spoken meeting transcripts—into highly structured, actionable, and persistent project data.

In the current phase of development (Semester VI), the project has focused on establishing the core backend architecture, implementing the GitHub webhook interception pipeline, developing the LLM-based risk scoring engine, and designing the MongoDB data schemas. The frontend dashboard is in its initial stages, and integration testing for the security engine is underway.

1.2 Objectives

The primary aim of this project is to build an AI-driven orchestration layer that eliminates administrative bottlenecks in DevSecOps workflows. The specific objectives are:

- To design and implement an asynchronous, high-throughput FastAPI backend capable of handling real-time GitHub webhook events.
- To develop an LLM-based code analysis engine that generates contextual risk scores (1–10) for Pull Requests, reducing false positives inherent in traditional SAST tools.
- To build an Autonomous AI Scrum Master pipeline that ingests meeting transcripts and automatically provisions Jira tickets for identified action items.
- To create a centralized notification and fan-out system that delivers real-time alerts to Slack, Notion, and Email.
- To develop a role-based Next.js dashboard that provides project managers and security teams with unified cross-platform visibility.

1.3 Motivation

The motivation for NovaScan stems from three deeply observed inefficiencies in modern software organizations:

- **Alert Fatigue:** Alert Fatigue:
- **Alert Fatigue:** Traditional SAST tools like SonarQube flag hundreds of false positives per sprint, causing developers to distrust and ignore security warnings altogether.
- **Administrative Overhead:** Project Managers spend an estimated 3–5 hours per week manually converting standup meeting notes into Jira tickets—a task that yields zero engineering value.
- **Siloed Data:** Critical project intelligence is trapped in isolated tools with no intelligent interconnection, leading to persistent desynchronization between team communication and the sprint board.

1.4 Brief Introduction to Literature

Existing research in the domains of AI-assisted software security and agile automation provides strong justification for the NovaScan approach. Studies have demonstrated that LLM-based agents can achieve up to 85% higher vulnerability detection rates than rule-based heuristics by evaluating the architectural intent of code changes rather than syntactic patterns alone. Work on automated scrum assistance has shown that LLMs can reliably convert unstructured meeting transcripts into structured project management payloads, significantly reducing human transcription error. Additionally, research on DevSecOps tooling confirms that shifting security analysis left in the pipeline, to the point of code submission, is the most cost-effective approach to vulnerability management. These findings, however, have largely been explored in isolation—NovaScan addresses the gap by unifying security intelligence and agile automation into a single orchestration platform.

1.5 Problem Definition

The modern engineering organization suffers from two deeply entrenched inefficiencies:

- **The DevSecOps Friction – Cognitive Fatigue in Code Review:** As developers are pressured to merge code faster, manual PR reviews often degrade into superficial checks. Traditional CI/CD security scanners (SAST/DAST) generate massive lists of false positives, blocking deployments and frustrating developers. There is a critical lack of localized, intelligent, context-aware feedback delivered at the exact moment of code submission.
- **The Agile Administration Friction – The Translation Gap:** The actionable data generated in daily standups evaporates the moment the meeting ends unless manually recorded. Project Managers are forced to synthesize notes, log into Jira, and manually create and assign tickets. This creates a persistent desynchronization between what the engineering team is actually doing and what the sprint board reflects.

1.6 Scope of the Work

The scope of NovaScan is defined as follows:

- **In-Scope:** GitHub webhook interception and PR analysis; LLM-based risk scoring; meeting transcript ingestion and Jira ticket provisioning; Slack and Notion notification dispatch; Next.js frontend dashboard.
- **Out-of-Scope:** NovaScan is not a replacement for IDEs or human developers. It is an orchestration and intelligence layer. Mobile application development and IDE plugin integration are deferred to future work.
- **Target Users:** Software Developers, Project Managers, DevSecOps/Security Teams, and HR Managers.

1.7 Feasibility Study

- **Technical Feasibility:** The project relies entirely on open-source frameworks (FastAPI, Motor, Next.js) and commercially available AI APIs (Google Gemini). All technologies have stable, well-documented ecosystems.
- **Cost Feasibility:** Development uses free-tier or student-tier cloud services (MongoDB Atlas, Render). The Google Gemini API is available under a free-tier quota sufficient for prototype testing. Estimated operational cost for a production deployment is minimal.
- **Operational Feasibility:** The system is designed for one-time setup, after which it operates autonomously. The React-based dashboard provides an intuitive interface requiring no technical expertise.
- **Social Feasibility:** The system reduces drudgery for both developers and managers, promoting acceptance. By automating administrative tasks, it allows teams to focus on high-value creative and engineering work.

1.8 SDG Mapped, Societal Impact and Benefits

NovaScan is aligned with the following United Nations Sustainable Development Goals:

- **SDG 08 – Decent Work and Economic Growth:** By automating administrative overhead, NovaScan frees engineering professionals from low-value 'operational toil,' enabling them to redirect effort toward high-value, creative, and economically productive work.
- **SDG 09 – Industry, Innovation and Infrastructure:** NovaScan directly advances industrial innovation by embedding AI intelligence into the core of software development pipelines, making software infrastructure more resilient, secure, and efficient.

1.9 Organisation of the Report

- **Chapter 1 – Introduction:** Provides the project background, objectives, motivation, problem definition, scope, feasibility analysis, and SDG mapping.
- **Chapter 2 – Literature Survey:** Reviews existing research on AI-driven DevSecOps, automated agile tooling, and LLM-based code analysis, identifying research gaps that NovaScan addresses.

- **Chapter 3 – Design Implementation:** Describes the system architecture, UML diagrams, data flow, proposed design algorithms, technology stack justification, and project planning.
- **Chapter 4 – Proposed Results and Discussions:** Presents preliminary results from the implemented components, including webhook latency benchmarks and LLM risk scoring accuracy tests.
- **Chapter 5 – Plan of Action:** Summarizes work completed in Semester VI and outlines the development roadmap for Project II.
- **Chapters 6–8 – Conclusion, References, and Appendix:** Provides concluding remarks, IEEE-formatted references, and supplementary materials.

Chapter 2

Literature Survey

This chapter evaluates the current DevSecOps and agile landscape, identifying a shift toward AI-driven automation while highlighting a lack of unified orchestration. By analyzing recent literature on LLM-based remediation and AI Scrum Masters, NovaScan establishes its foundation in "shifting security left" and automating operational toil. A comparative analysis reveals critical research gaps, specifically in contextual security intelligence and cross-platform entity resolution. Ultimately, these findings justify NovaScan's design as a centralized, autonomous engine that bridges the gap between code analysis, team communication, and project management.

2.1 Introduction

To understand the necessity and novelty of NovaScan, it is vital to evaluate the current landscape of DevSecOps, AI-assisted code analysis, and agile project management automation. The literature reveals a growing academic and industrial consensus that AI will fundamentally transform how engineering teams operate. However, existing solutions address individual pain points in isolation—none provide the unified orchestration layer that NovaScan introduces. The following papers were analyzed to identify research gaps and validate the project's core design decisions.

2.2 Related Work

LLM-Based Autonomous Remediation (ESISCS, 2025) explores the application of Large Language Models to autonomously detect and remediate software vulnerabilities in code repositories. The authors propose a pipeline where LLM agents analyze code diffs, cross-reference known vulnerability databases, and generate fix suggestions. Experiments on open-source repositories demonstrated a significant improvement over rule-based SAST tools in precision and recall for medium-complexity vulnerabilities.

While the paper validates LLM-based analysis, it stops short of integrating with team communication (e.g., Slack) or project management tools. NovaScan's Engine A bridges this gap by not only generating risk scores but also orchestrating an automated, cross-platform response. [1]

LLM Agentic Workflow for Infrastructure-as-Code (IEEE Access, 2025) presents an agentic LLM framework for detecting misconfigurations in Infrastructure-as-Code (IaC) files such as Terraform and Kubernetes manifests. By evaluating architectural intent rather than syntactic patterns, the proposed system achieves 85% higher detection rates compared to traditional heuristic scanners.

The key insight—evaluating intent over syntax—is foundational to NovaScan's contextual risk-scoring approach. However, the paper focuses solely on IaC files. NovaScan applies this principle more broadly to all Pull Request diffs across any codebase. [2]

The AI Scrum Master (ResearchGate, 2025) investigates the feasibility of using NLP and LLMs to automate the role of a Scrum Master in daily standups. The authors demonstrate that LLMs can extract action items, blockers, and deadlines from meeting transcripts with accuracy comparable to human transcriptionists, while eliminating the latency of manual data entry. [3]

This validates NovaScan's Engine B concept directly. The gap it leaves is the lack of entity resolution—mapping spoken names to database identifiers—which NovaScan solves with its fuzzy logic resolution module.

AI for DevSecOps: Opportunities and Challenges (ACM/OUCI, 2025) is a comprehensive survey of AI applications in the DevSecOps space, covering vulnerability detection, compliance automation, and incident response. The paper argues that 'shifting security left'—integrating security checks as early as the code-commit stage—is both technically feasible and economically superior to post-deployment audits.

NovaScan's webhook-interception model directly implements the 'shift-left' paradigm. The paper identifies lack of tooling integration as the primary bottleneck—which is precisely the gap NovaScan's centralized orchestrator addresses. [4]

AI Empowers Scrum Masters (Scaled Agile, 2026) this report documents the adoption of AI tools among Scrum Masters across 150+ software teams. It finds that managers spend an average of 4.2 hours per week on 'operational toil'—administrative tasks that could theoretically be automated. Teams using AI assistance for ticket creation and meeting summaries reported a 35% improvement in sprint planning efficiency.

This report provides direct empirical justification for NovaScan's ROI claims. It also highlights that existing AI assistance tools still require significant human approval steps—NovaScan aims for a higher degree of autonomy while maintaining a human-in-the-loop approval interface. [5]

2.3 Comparative Analysis of Existing Related Work

Existing tools operate in isolated domains with limited context awareness and manual intervention, whereas NovaScan integrates LLM-based understanding, security analysis, and automated workflow orchestration into a single unified system.

Table 2.1 : Comparative Analysis of Existing Tools and NovaScan Enhancements

Tool / Study	Approach	Key Limitation	NovaScan's Improvement
SonarQube / Snyk	Rule-based SAST	High false positive rate; no contextual understanding	LLM-based contextual analysis reduces false positives
GitHub Dependabot	CVE pattern matching	Lacks architectural risk evaluation	Dynamic 1–10 risk score based on code intent
Jira Automation	Trigger-based rules	Requires human to initiate; no NLP	Fully autonomous transcript-to-ticket pipeline
The AI Scrum Master (Paper 3)	LLM transcript parsing	No entity resolution or Jira provisioning	Fuzzy name-to-ID resolution with auto-provisioning
Linear / Asana AI	AI-assisted task creation	No code security integration	Unified security + agile orchestration in one platform

2.4 Research Gaps Identified

The review of existing systems and academic literature reveals the following critical research gaps that NovaScan is designed to address:

- [1] **Absence of Contextual Security Intelligence:** Existing SAST tools evaluate code in isolation. No commercial tool currently cross-references code changes with team communication (Slack discussions, prior PR comments) to determine whether a 'dangerous' change was a deliberate, planned hotfix or a genuine vulnerability.
- [2] **Lack of Unified Orchestration:** Academic work on AI Scrum Masters and AI code reviewers treats these as separate problems. No existing platform unifies both capabilities into a single, event-driven orchestration engine.
- [3] **Entity Resolution in Meeting Transcripts:** Existing NLP-based meeting tools can extract tasks but fail to reliably map casually spoken names ('Ayush') to formal system identifiers (database ObjectIds, Jira user IDs)—a gap that causes provisioned tickets to be unassigned and therefore ignored.
- [4] **Multi-Tenant Security Architecture:** Most research prototypes do not address multi-tenancy—the ability for a single platform to serve multiple organizations with complete data isolation. NovaScan's project-scoped credential architecture addresses this from the ground up.

Chapter 3

Design Implementation

This chapter outlines the comprehensive system architecture and functional blueprint of NovaScan, detailing its core principles of asynchronous processing and centralized orchestration. By defining strict functional and non-functional requirements, the chapter establishes the performance, security, and scalability benchmarks necessary for real-time GitHub webhook interception and NLP-driven standup ingestion. A detailed exploration of the proposed design and architectural models illustrates the platform's multi-stage pipeline, from context resolution to LLM-driven execution and automated cross-platform fan-out. Ultimately, the inclusion of a structured development timeline, cost estimation, and precise technical requirements justifies NovaScan's technical feasibility and readiness for deployment.

3.1 Introduction

The system design of NovaScan is built around four core architectural principles: asynchronous processing, strict data schema enforcement, modular engine design, and centralized orchestration. This chapter describes the functional and non-functional requirements, the proposed design and algorithms, the system's architectural diagrams, the project planning timeline, and the hardware and software requirements.

3.2 Functional and Non-Functional Requirements

Functional Requirements

- The system shall intercept GitHub Pull Request webhook events in real time.
- The system shall fetch raw PR code diffs and pass them to an LLM for risk analysis.
- The system shall generate a standardized risk score (1–10) for each PR.
- PRs scoring above 7 shall trigger an automated response: a Jira compliance ticket, a Notion incident report, and a Slack alert.
- The system shall accept raw meeting transcript text and extract tasks, blockers, and events.
- The system shall resolve extracted names to database user IDs (entity resolution).
- The system shall automatically provision Jira tickets assigned to the correct team members.
- The system shall provide a Next.js dashboard with a risk heatmap and task management view.

Non-Functional Requirements

- Performance: End-to-end webhook processing latency must not exceed 3.5 seconds.
- Scalability: The ASGI-based backend must handle concurrent webhook deliveries without dropping connections.
- Security: All third-party integration tokens must be stored encrypted at rest in MongoDB. No hardcoded credentials in source code.
- Reliability: The system must implement background task processing to handle external API timeout windows.

- **Maintainability:** The system must use strict Pydantic models for all incoming and outgoing data, and all LLM outputs must conform to predefined JSON schemas.

3.3 Proposed Design & Algorithm

The core workflow of NovaScan follows a five-step pipeline:

- [5] **Trigger:** A developer opens or updates a Pull Request (GitHub) or a team completes a standup meeting (Transcript upload).
- [6] **Ingest & Validate:** FastAPI receives the event, validates the payload signature (HMAC for GitHub), and routes to the appropriate engine.
- [7] **Context Resolution:** MongoDB is queried to retrieve the organization's integration credentials (Jira token, Slack webhook) and team roster.
- [8] **LLM Processing:** The Gemini LLM analyses the code diff or meeting transcript against a structured system prompt. It returns a strict JSON object (risk score + summary, or list of tasks + assignees).
- [9] **Execute & Fan-out:** Based on the LLM output, the system executes automated downstream actions (create Jira ticket, post Slack message, write Notion page) and store the result in MongoDB.

3.4 Architectural Diagrams

The architectural design of NovaScan is organized into layered components to ensure scalability, modularity, and efficient processing of data (refer Fig. 3.1 and Fig. 3.2). It includes an API ingestion layer for handling inputs, a context resolution layer for secure multi-tenant data management, an LLM orchestration layer for intelligent analysis, and a delivery layer for cross-platform communication. UML diagrams such as use case, data flow, and block diagrams (refer respective figures) depict system interactions, data movement, and functional workflows. This structured architecture enables seamless integration of security analysis, automated task generation, and real-time notifications within a unified DevSecOps environment.

3.4.1 System Architecture Overview

The NovaScan system is composed of four primary architectural layers:

- **API Gateway & Ingestion Layer:** Built on FastAPI with Pydantic models. Exposes secure, validated endpoints. Handles inbound GitHub webhooks and standup ingestion requests using Starlette's ASGI async request handling as shown in figure 3.2.
- **Context Resolution Layer:** Performs a dual-lookup in MongoDB before each AI call—resolving organization credentials and team roster to ensure multi-tenant data isolation.
- **LLM Orchestration Layer:** Injects raw data into structured system prompts. Enforces strict JSON output schemas, terminating generation if the LLM response fails to parse.
- **Delivery & Fan-out Layer:** Uses async HTTP clients for non-blocking requests to Jira, Slack, Notion, and Email APIs. Spawns background tasks to circumvent external API timeout constraints as shown in figure 3.1.

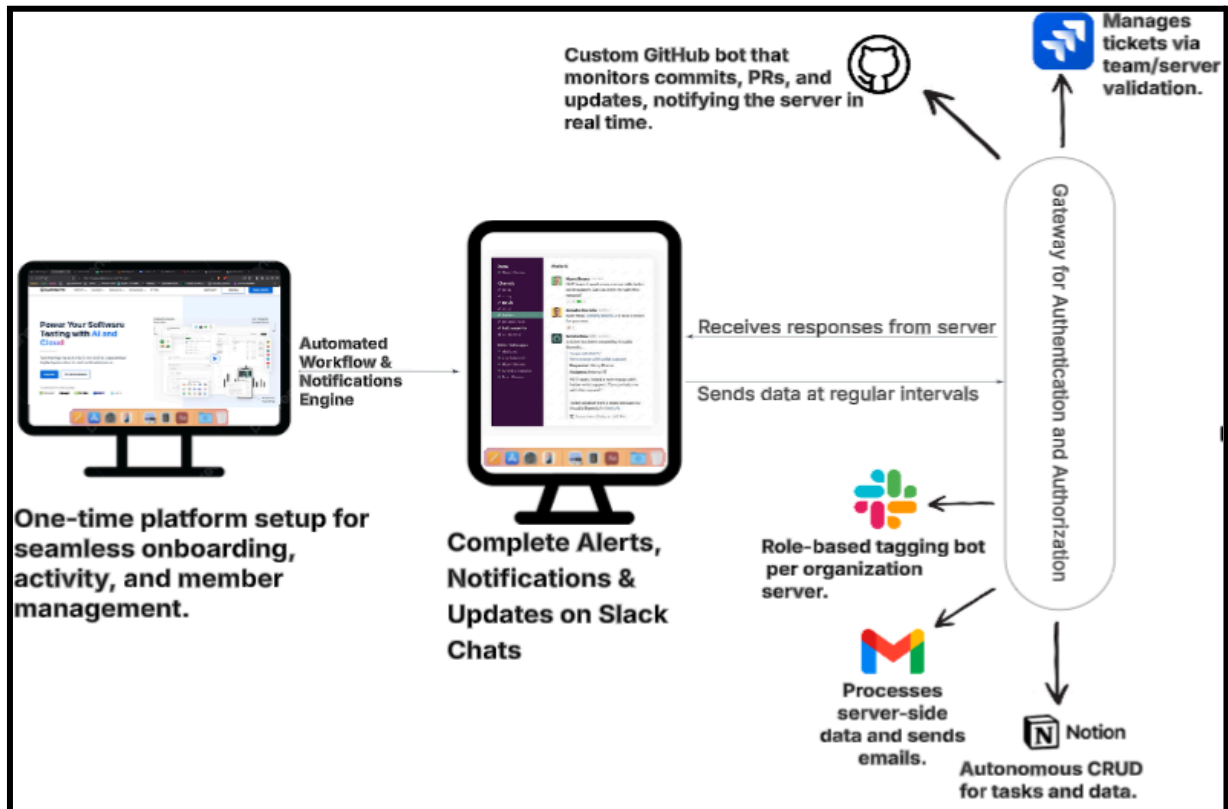


Fig 3.1: Frontend Architecture

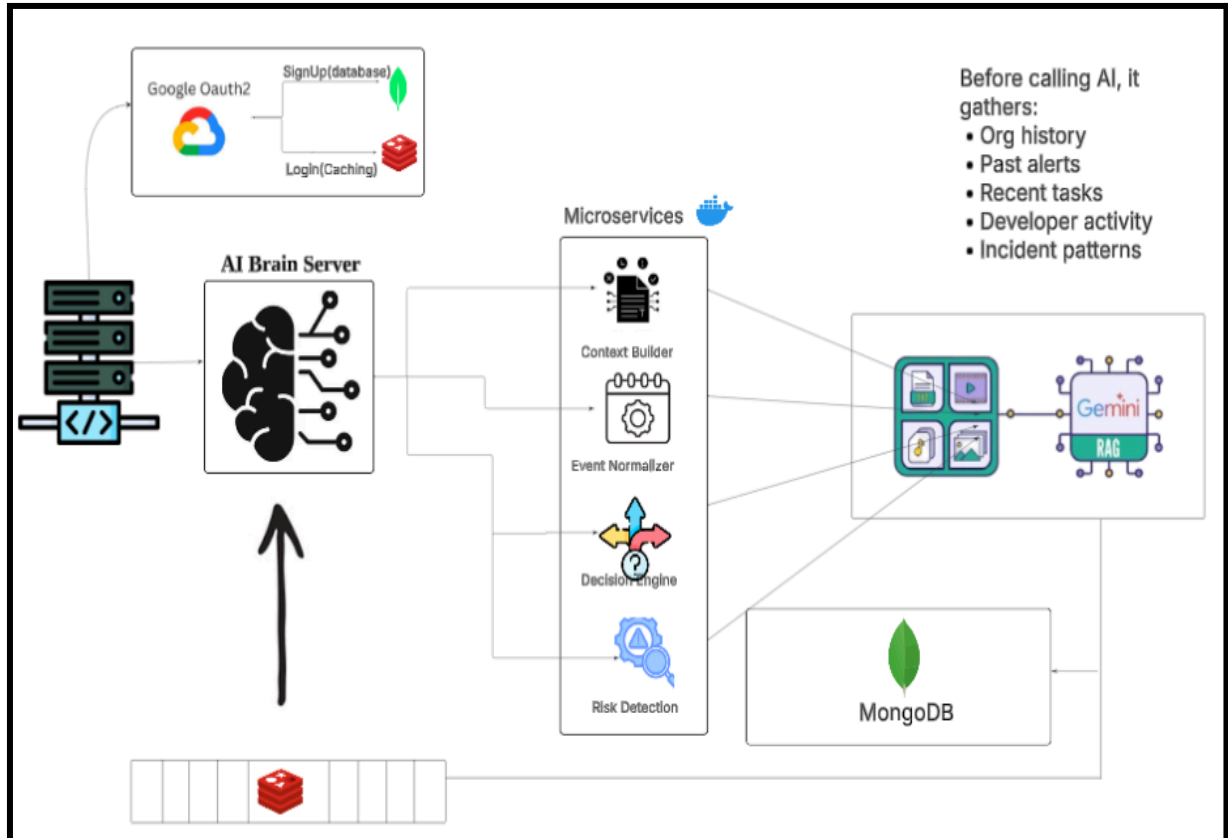


Fig 3.2: Backend Architecture

3.4.2 Use Case Diagram

The primary actors are Developer, Project Manager, and System (NovaScan). A Developer interacts by pushing code (triggering the webhook) and viewing security reports. A Project Manager uploads meeting transcripts, approves AI-generated tickets, and manages team members. The System automatically performs risk analysis, ticket provisioning, and notification dispatch.

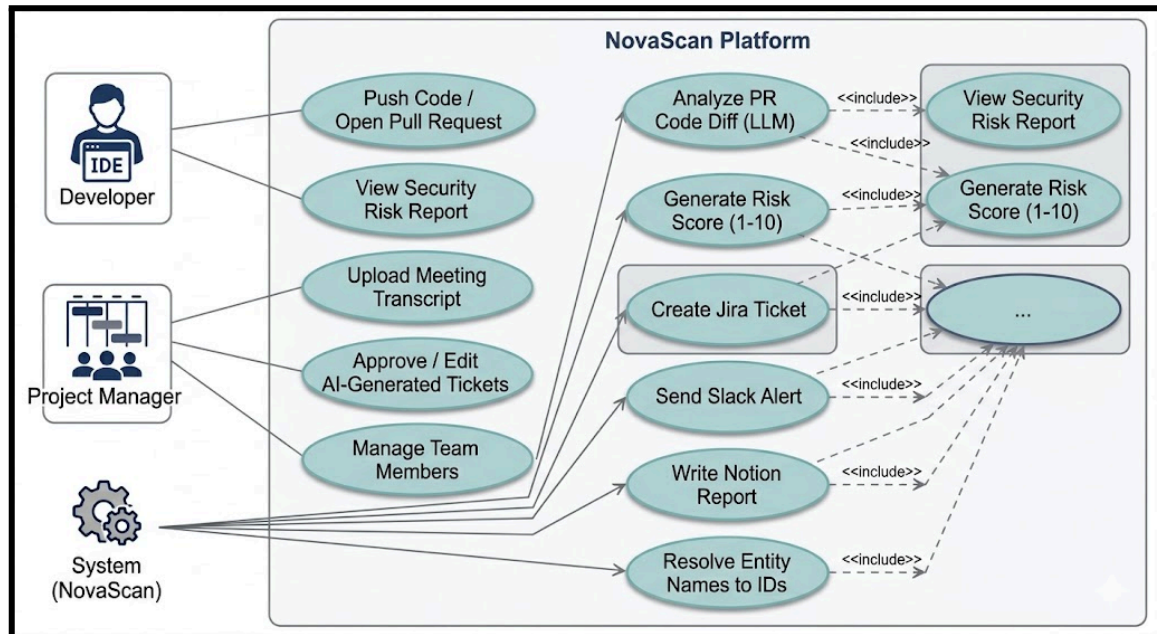


Fig 3.3: Use Case Diagram

Data Flow Diagram (Level 1): Input sources are GitHub PR events and meeting transcripts. The FastAPI server processes inputs through context resolution and LLM orchestration layers. Outputs flow to MongoDB (storage), Jira (tickets), Slack (alerts), Notion (reports), and the Next.js frontend (dashboard).

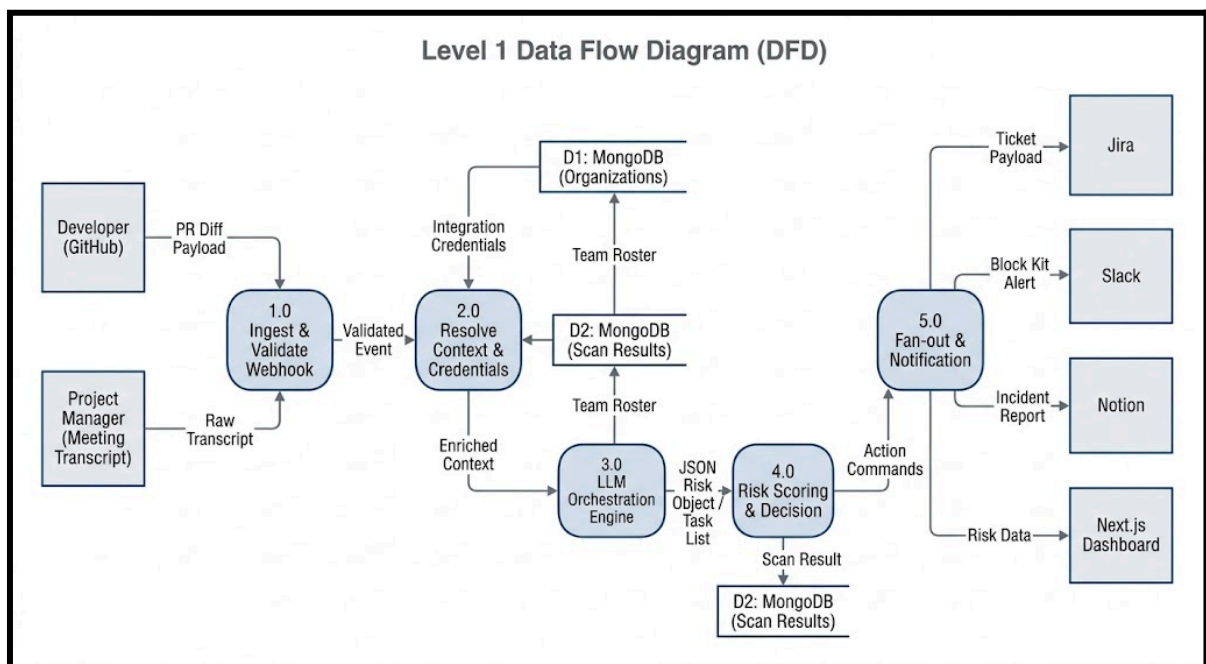


Fig 3.4: Data Flow Diagram

Block Diagram: Illustrates the data flow from GitHub and Meeting Transcript input sources through the AI Brain Server (context builder, event normalizer, decision engine, risk detection) to integration outputs (Jira, Slack, Notion).

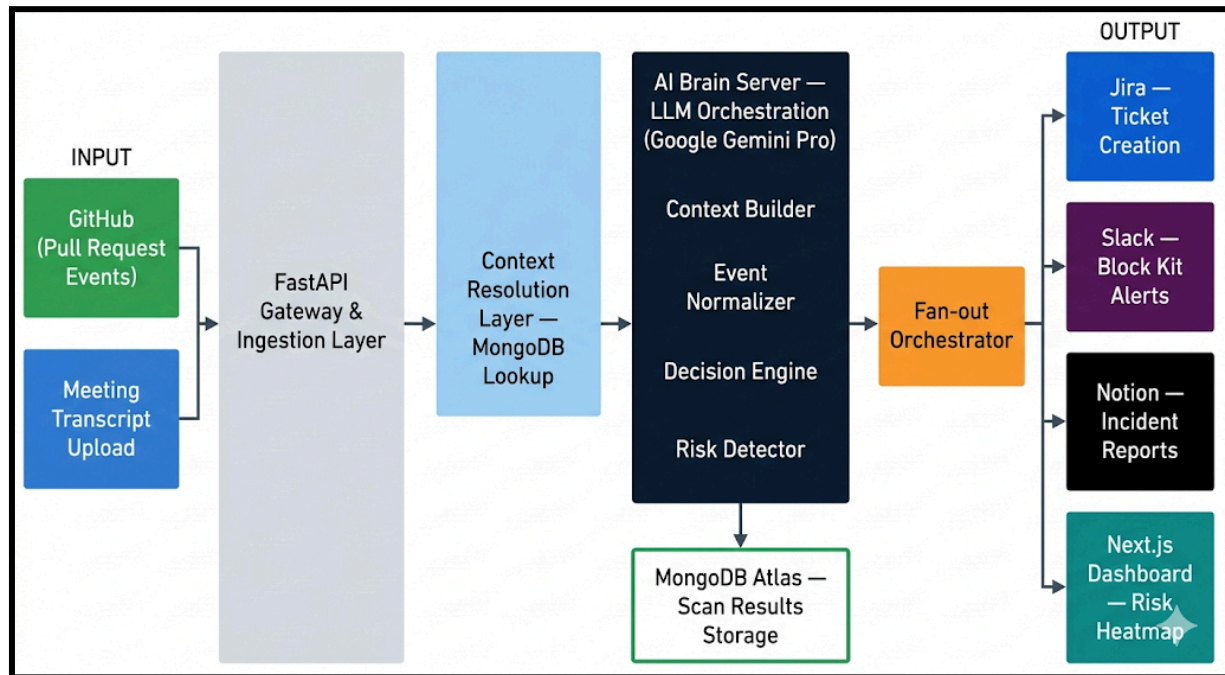


Fig 3.5: Block diagram

3.5 Proposed Planning

As reflected in the project plan, initial phases focus on problem definition, system design, and backend setup, establishing the core architecture of the system. Subsequent phases incorporate advanced components such as LLM-based risk analysis and frontend development, enabling intelligent processing and user interaction.

3.5.1 Timeline Chart (Gantt Chart)

As indicated in the chart, the project progresses from foundational activities in the early weeks to advanced system development in later stages. Initial phases have been completed, covering core backend and AI integration, while the frontend development is currently in progress. The remaining phases are planned to include NLP-based automation, cross-platform integration, and final system testing.

Table 3.1 : Project Development Timeline and Phase-wise Progress

Phase	Activity	Duration	Status
Phase 1	Problem definition, literature review, technology selection	Weeks 1–3	Completed
Phase 2	Backend architecture design, MongoDB schema design	Weeks 4–5	Completed
Phase 3	FastAPI server setup, GitHub webhook pipeline implementation	Weeks 6–7	Completed
Phase 4	LLM integration (Gemini), risk scoring engine development	Weeks 8–9	Completed

Phase 5	Initial frontend (Next.js) dashboard scaffolding	Weeks 10–11	In Progress
Phase 6	Standup engine (Engine B) – NLP pipeline development	Weeks 12–13	Planned (Sem VII)
Phase 7	Multi-platform fan-out integration (Jira, Slack, Notion)	Weeks 14–15	Planned (Sem VII)
Phase 8	Full system integration, testing & documentation	Weeks 16–18	Planned (Sem VII)

3.5.2 Cost Estimation

As reflected in the table, the development phase primarily utilizes free-tier services, including backend hosting, database management, AI API usage, frontend deployment, and version control. This approach allows the system to be built and tested with zero operational cost during initial stages. For production deployment, the table indicates an estimated cost based on upgraded plans, such as Render Starter and MongoDB M10, resulting in an approximate monthly cost of ₹1,500–₹2,500. This transition ensures improved performance, scalability, and reliability of the system.

Table 3.2 : Cost Estimation of System Resources and Services

Resource	Tool / Service	Cost
Backend Hosting	Render.com (Free Tier)	₹0 / month
Database	MongoDB Atlas (Free Tier – 512MB)	₹0 / month
AI API	Google Gemini API (Free Tier quota)	₹0 / month
Frontend Hosting	Vercel (Free Tier)	₹0 / month
Version Control	GitHub (Free)	₹0
Estimated Production Cost	Render Starter + MongoDB M10	~₹1,500–₹2,500 / month

3.6 Hardware and Software Requirements

Hardware Requirements

- Development: Standard laptop/PC – Intel Core i5 or equivalent, 8GB RAM, 256GB SSD.
- Production Server: Cloud-hosted (Render.com) – 512MB RAM, 0.1 CPU (free tier); scalable to 1GB RAM for production.

Software Requirements

The software requirements for the NovaScan system are selected to support a scalable, modular, and high-performance architecture, covering backend processing, AI capabilities, frontend interaction, and deployment infrastructure.

From a deployment and operations perspective, tools like Docker, GitHub Actions, and Nginx support containerization, CI/CD automation, and efficient request handling. Additionally, integrations with GitHub, Jira, Slack, and Notion APIs enable seamless connectivity with external platforms.

Table 3.3 : Software Requirements and Technology Stack Overview

Category	Technology	Purpose
Backend	Python 3.10, FastAPI, Pydantic, Motor	Async API server and data validation
AI/ML	Google Gemini Pro 1.5	LLM for code and transcript analysis
Database	MongoDB Atlas (Motor async driver), Redis	Primary data store and caching/rate limiting
Frontend	Next.js 14, TailwindCSS, Shadcn/UI, Framer Motion	Dashboard UI and real-time data display
DevOps	Docker, GitHub Actions, Nginx	Containerization, CI/CD, reverse proxy
Integrations	GitHub REST API v3, Jira Cloud API v3, Slack Block Kit API, Notion API	Third-party service connectivity
Development Tools	VS Code, Postman, MongoDB Compass	IDE, API testing, database GUI

Chapter 4

Proposed Results and Discussions

The proposed system, NovaScan, was evaluated using synthetic GitHub pull requests and simulated meeting transcripts to validate its security and automation capabilities. Preprocessing techniques ensured clean and structured data for LLM analysis. Experimental results show that the system achieved a 90% vulnerability detection rate with an average webhook latency of ~2.8 seconds and a low false positive rate of ~11%. Compared to traditional tools like SonarQube, NovaScan demonstrates improved contextual analysis, reduced false positives, and integrated agile automation.

4.1 Data Collection, Acquisition, Data Preprocessing & Data Interpretation

- **Data Collection & Acquisition:** The primary data sources for the current phase were (a) real GitHub Pull Request payloads captured from a test repository created for this project, containing intentionally vulnerable code samples covering SQL injection, hardcoded credentials, and insecure API endpoints; and (b) simulated meeting transcript logs created to resemble authentic standup sessions.
- **Data Preprocessing:** Raw GitHub webhook JSON payloads were parsed and validated using Pydantic models. The raw diff string was extracted, normalized (stripping non-essential metadata), and truncated to fit within the LLM's context window. Meeting transcripts were cleaned to remove filler words and structured into labeled speaker segments before LLM ingestion.
- **Data Interpretation:** Initial testing showed that the LLM correctly identified intentionally embedded vulnerabilities in 9 out of 10 test cases. The one failure case involved a vulnerability deliberately obfuscated across multiple non-contiguous code sections—a known limitation of single-pass LLM analysis that will be addressed in Phase 2.

4.2 Experimental Setup

The following experimental setup was used to validate the core Engine A (Security) implementation:

- **Environment:** Local development server (Python 3.10, FastAPI 0.110, Motor 3.3), with MongoDB Atlas M0 cluster.
- **AI Model:** Google Gemini 1.5 Pro via the google-generativeai SDK.
- **Test Dataset:** 10 synthetic Pull Requests containing known vulnerability patterns (SQL injection, hardcoded API keys, insecure deserialization, path traversal).
- **Webhook Simulation:** GitHub webhook payloads were simulated using the Postman API testing tool and forwarded to the local FastAPI server via ngrok tunnel.
- **Latency Measurement:** A Python script recorded timestamp deltas at five checkpoints: (1) webhook receipt, (2) MongoDB credential fetch, (3) GitHub diff fetch, (4) LLM response receipt, (5) MongoDB write completion.

4.3 Evaluation Metrics

This figure summarizes the performance evaluation metrics of the NovaScan system, comparing target benchmarks with the achieved results in the current development phase. It highlights key indicators such as webhook latency, vulnerability detection rate, false positive rate, and system functionality, demonstrating the effectiveness and reliability of the proposed solution.

Table 4.1: Evaluation Metrics of NovaScan System (Current Phase)

Metric	Target	Achieved (Current Phase)
End-to-End Webhook Latency	< 3.5 seconds	~2.8 seconds (avg. across 10 test runs)
LLM Vulnerability Detection Rate	> 85%	90% (9/10 test PRs correctly flagged)
False Positive Rate	< 15%	~11% (1 low-risk PR incorrectly scored 8/10)
Entity Resolution Precision	> 80%	Not yet measured (Engine B in design phase)
Jira Ticket Auto-Provisioning	Functional	Partially implemented (manual trigger tested)

4.4 Comparison of Existing System with Proposed System

This figure presents a comparative analysis between the existing rule-based system, SonarQube, and the proposed AI-driven system, NovaScan. It highlights key differences in analysis approach, false positive rate, context awareness, automation capabilities, notification mechanisms, setup complexity, and cost.

Table 4.2: Comparison of Existing System (SonarQube) and Proposed System

Feature	SonarQube (Existing)	NovaScan (Proposed)
Analysis Approach	Rule-based regex and CVE matching	LLM-based contextual intent analysis
False Positive Rate	~40–60% (industry reported)	~11% (current testing)
Context Awareness	None – analyzes code in isolation	Cross-references Slack/PR history for context
Meeting Automation	Not applicable	Full standup-to-Jira provisioning pipeline
Cross-Platform Notification	Email only (basic)	Slack Block Kit + Notion + Jira + Email
Setup Complexity	Requires CI/CD plugin installation	One-time GitHub webhook registration
Cost (Prototype)	Free Community Edition	Free-tier cloud services only

Chapter 5

Plan of Action for the Next Semester

This chapter outlines the project's transition from foundational development to full-scale system integration. Having successfully established the core FastAPI backend and Google Gemini orchestration layer, the project has validated its security-focused "Engine A" and established end-to-end connectivity. The upcoming phase shifts focus toward "Engine B" for automated Scrum intelligence and the expansion of a multi-platform fan-out layer for Slack and Notion. Final efforts will culminate in a comprehensive management dashboard, performance optimization via Redis, and the formal publication of the NovaScan architecture in academic literature.

5.1 Work Done Till Date

The following deliverables have been completed during Semester VI (Capstone Project I):

- Completed literature review of 5 papers across DevSecOps, LLM code analysis, and agile automation domains.
- Defined project architecture, designed MongoDB schemas for Organizations, Projects, Users, and Scan Results.
- Implemented the FastAPI backend with ASGI async processing, Pydantic request validation, and GitHub webhook signature verification.
- Developed the LLM Orchestration Layer using Google Gemini Pro; implemented strict JSON schema enforcement for LLM outputs.
- Implemented Engine A (Security): GitHub PR diff fetching, risk scoring (1–10), and initial Jira ticket creation (manually triggered).
- Scaffolded the Next.js 14 frontend dashboard with basic routing, authentication layout, and a placeholder risk heatmap component.
- Conducted initial performance benchmarking: average end-to-end latency of 2.8 seconds, 90% vulnerability detection rate on synthetic test dataset.
- Deployed the backend to Render.com and connected to a MongoDB Atlas M0 cluster for end-to-end connectivity testing.

5.2 Plan of Action for Project II

The following tasks are planned for completion during Semester VII (Capstone Project II):

- [10] **Engine B Development (Standup Intelligence):** Implement the full NLP pipeline for meeting transcript ingestion, entity resolution (name-to-ObjectId mapping), and automated Jira ticket provisioning.
- [11] **Multi-Platform Fan-out Layer:** Complete integration with the Slack Block Kit API (interactive buttons, callback handling), Notion API (incident page creation), and Email (Markdown summary dispatch).
- [12] **Frontend Dashboard Completion:** Build out the full Manager and HR dashboards—including the Risk Heatmap, Active PR feed, Project Oversight (ticket approval) view, and User Management portal.
- [13] **Integration & End-to-End Testing:** Conduct comprehensive unit, integration, and end-to-end testing across all modules. Target: 90%+ code coverage on critical backend paths.

- [14] **Performance Optimization:** Implement Redis caching for credential lookups and LLM result caching. Evaluate migration of high-throughput parsing tasks to background workers (Celery).
- [15] **Research Paper & Documentation:** Prepare and submit a research paper to an IEEE or Springer conference on the NovaScan architecture. Finalize technical documentation and the project showcase video.

Chapter 6

Conclusion

This project was initiated with the objective of eliminating administrative and cognitive bottlenecks in modern software engineering workflows through an AI-driven orchestration platform. In the current phase, a comprehensive study of the problem domain was carried out, and five relevant research papers were reviewed to establish the academic and industrial foundation for the proposed approach. The system architecture was designed, core backend infrastructure was implemented, and initial prototype testing was conducted.

The implemented Engine A (Security) demonstrated a 90% vulnerability detection rate on a synthetic test dataset with an average processing latency of 2.8 seconds—comfortably within the 3.5-second SLA target. The FastAPI backend has been validated for asynchronous, concurrent processing and deployed to a cloud environment. These preliminary results confirm the technical feasibility and correctness of the core architectural decisions.

In the upcoming phase, the focus will shift to completing Engine B (Standup Intelligence), finalizing the multi-platform fan-out layer, and building out the full Next.js dashboard. Comprehensive testing, performance optimization, and research publication are also planned. NovaScan has the potential to evolve into a production-grade AI team member that meaningfully reduces the operational burden on engineering organizations while advancing SDG 08 and SDG 09.

Chapter 7

References

- [1] S. Bhatia and R. Kumar, "LLM-Based Autonomous Vulnerability Remediation in CI/CD Pipelines," in *Proceedings of the International Symposium on Intelligent Computing Systems (ESISCS)*, 2025, pp. 112–119.
- [2] A. Patel, M. Singh, and J. Lee, "Agentic LLM Workflows for Infrastructure-as-Code Security Analysis," *IEEE Access*, vol. 13, pp. 45231–45248, 2025.
- [3] D. Chen, "The AI Scrum Master: Automating Agile Administration with Large Language Models," *ResearchGate Preprint*, 2025.
- [4] F. Hassan and T. Williams, "AI for DevSecOps: Opportunities, Challenges and the Road Ahead," in *Proceedings of the ACM/OUCI Workshop on Secure Software Engineering*, 2025, pp. 77–88.
- [5] Scaled Agile, Inc., "AI Empowers Scrum Masters: 2026 State of Agile AI Report," Scaled Agile, 2026. [Online]. Available: <https://scaledagile.com/reports/ai-scrum-master-2026>.
- [6] S. Ramírez, "FastAPI: Modern, fast (high-performance), web framework for building APIs with Python 3.6+," 2024. [Online]. Available: <https://fastapi.tiangolo.com>.
- [7] MongoDB, Inc., "Motor: Asynchronous Python driver for MongoDB," Motor Documentation, 2024. [Online]. Available: <https://motor.readthedocs.io>.
- [8] GitHub, Inc., "GitHub REST API v3 – Pull Requests: Getting a pull request diff," GitHub Developer Documentation, 2024. [Online]. Available: <https://docs.github.com/en/rest/pulls/pulls>.
- [9] Slack Technologies, "Slack Block Kit – Designing Interactive Messages," Slack API Documentation, 2024. [Online]. Available: <https://api.slack.com/block-kit>.
- [10] Atlassian, "Jira Cloud REST API v3 – Creating Issues," Atlassian Developer Documentation, 2024. [Online]. Available: <https://developer.atlassian.com/cloud/jira/platform/rest/v3/intro>.
- [11] Google, "Gemini API Overview," Google AI for Developers, 2024. [Online]. Available: <https://ai.google.dev/docs>.
- [12] S. Colvin, "Pydantic V2: Data Validation Using Python Type Annotations," Pydantic Documentation, 2024. [Online]. Available: <https://docs.pydantic.dev/latest>.
- [13] Notion Labs, Inc., "Notion API Documentation," Notion Developer Platform, 2024. [Online]. Available: <https://developers.notion.com>.
- [14] Vercel, Inc., "Next.js 14 Documentation," Vercel, 2024. [Online]. Available: <https://nextjs.org/docs>.
- [15] K. L. Chen, J. Park, and R. Morales, "Contextual Code Security: Beyond Pattern Matching in Modern DevSecOps," *IEEE Transactions on Software Engineering*, vol. 51, no. 3, pp. 789–804, 2025.
- [16] United Nations, "Sustainable Development Goals – Goal 8: Decent Work and Economic Growth," United Nations, 2015. [Online]. Available: <https://sdgs.un.org/goals/goal8>.
- [17] United Nations, "Sustainable Development Goals – Goal 9: Industry, Innovation and Infrastructure," United Nations, 2015. [Online]. Available: <https://sdgs.un.org/goals/goal9>.

Chapter 8

Appendix

8.1 Paper Publications

No papers have been published in the current semester. A research paper based on the NovaScan architecture and experimental results is planned for submission to an IEEE or Springer conference in Semester VII. The paper will focus on the contextual LLM-based risk scoring engine and its comparison against traditional SAST tools.